

SIFIRDAN BAŞLANGIÇ REHBERİ

Claude Code ile Çalışmaya Başla

Hiç bilmeyene; ilk oturumdan günlük akışa, kavramlardan hata kurtarmaya kadar resimli el kitabı.

🕒 İlk oturum 5 dakika · Tüm rehber ~25 dakika

Bu rehber kime, nereden başlamalı?

7 bölüm. Sırayla okumak en iyisi ama acelen varsa rolüne göre atla.

Hiç bilmeyen

Böl. 1 → 2 → 3
İlk oturum +
kavram + akış

Kullanan

Böl. 4 → 5
Cheat-sheet +
örnek oturum

Takılan

Böl. 6 → 7
Hata kurtarma +
sınırlar/SSS

#	Bölüm	Ne öğrenirsin
1	Claude Code nedir + ilk oturum	Aracı tanı, 5 dakikada ilk konuşma
2	En önemli kavram: Claude hatırlamaz	Neden journal/kural/hook var
3	Günlük akış (oturum döngüsü)	Sabah → iş → akşam ritüeli
4	Cheat-sheet — ne yazarsam ne olur	Tek sayfa komut kartı
5	Gerçek örnek oturum	Baştan sona bir gün, taklit et
6	Bir şey ters gitti — kurtarma	Panik anı çözümleri
7	Sınırlar, güven, SSS + sözlük	Claude neyi yapar/yapmaz, jargon

Altın kural baştan: Bu sistemde her şeyi ezberlemene gerek yok. Ne istediğini düz Türkçe söyle — doğru aracı Claude seçer, sana sorar, sen onaylarsın.

1 · Claude Code nedir + ilk oturum

Claude Code nedir?

Terminalde (komut satırında) çalışan bir **yapay zekâ yazılım asistanı**. Senin proje klasöründe oturur; kod okur, yazar, çalıştırır, git ile commit eder — ama **her önemli adımda sana sorar**. Chatbot değil; gerçek dosyalarınla çalışan bir takım arkadaşı.

Önce kurulu mu? Terminal aç, `claude --version` yaz. Numara dönerse kurulu. Dönmezse BT/kurulum ekibine danış — bu rehber kurulu olduğunu varsayar.

İlk oturum — 4 adım

```
# 1) Projenin klasörüne gir
cd D:\Dev\proje-adi

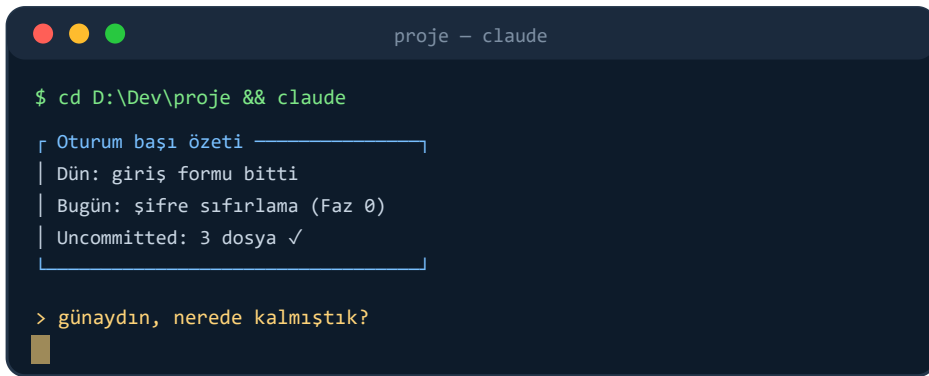
# 2) Claude'u başlat
claude

# 3) Açılınca otomatik "oturum başı özeti" gelir:
#   son commit'ler, yapılacaklar, dünden kalanlar

# 4) Sadece düz Türkçe yaz:
günaydın, nerede kalmıştık?
```

Hepsi bu. Claude dünkü notları okur, "şunu yapıyorduk, şuradan devam edelim mi?" der. **Kod bilmek şart değil** — ne istediğini anlat yeter.

Ekran neye benzer?



Terminal penceresi: açılışta otomatik özet, sonra düz Türkçe yazarsın.

Nasıl konuşulur?

İyi istek

"Giriş sayfasına 'şifremi unuttum' bağlantısı

Belirsiz istek

"Bir şeyler düzelt."

ekle."

Net hedef, Claude planlar + sana gösterir.

Claude ne istediğini sorar — zaman kaybı.
Hedefi söyle.

Çıkış: Bittiğinde *iyi geceler* yaz veya */handoff* . Claude bugünü not defterine yazar (sonraki bölümde neden önemli göreceksin). Sonra terminali kapatabilirsin.

2 · En önemli kavram: Claude hatırlamaz

Bu tek fikri anlarsan sistemin tamamı anlam kazanır.

Her oturum **sıfırdan** başlar. Dün konuştuklarınız, verdiğiniz talimatlar, aldığınız kararlar — Claude'un kafasında **kalmaz**. Tıpkı her sabah hafızası silinmiş çok yetenekli bir stajyer gibi.



Çözüm: hafızayı **dosyalara dışsallaştırmak**. Önemli her şey diske yazılır; her oturum başında Claude bunları okuyup "hatırlar". Üç tür hafıza dosyası:

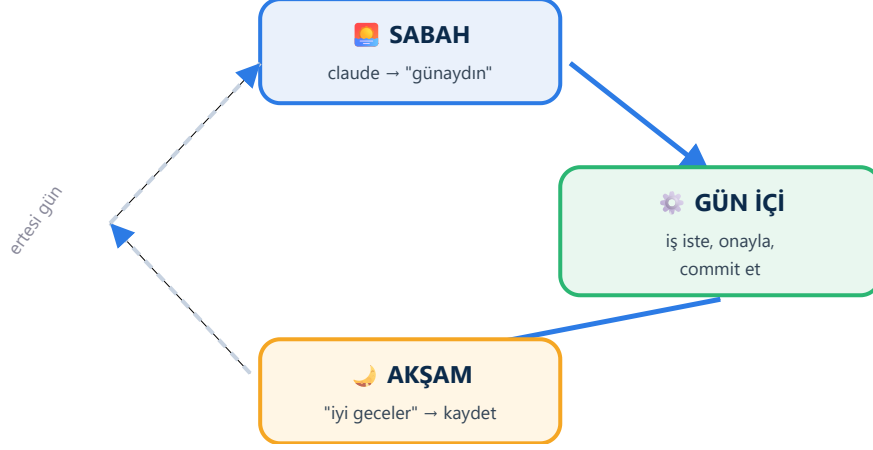
Dosya	Ne tutar	Örnek
Kurallar (.claude/rules)	Kalıcı davranış: "şöyle yap, şunu yapma"	"Sırları asla koda yazma"
Günlük (journal)	Her günün özeti: ne yapıldı, ne kaldı	"Bugün giriş bitti, yarın test"
Yapılacaklar (TODO.md)	Öncelikli iş listesi	"Faz 0: şifre sıfırlama"

Bundan çıkan en önemli alışkanlık: Yeni bir kural/karar söylediğinde "aklında tut" deme — işe yaramaz, buharlaşır. Bunun yerine **"bunu kurala yaz"** de. Claude dosyaya yazar, kalıcı olur.

Neden bu kadar önemli? Çünkü "Claude akıllı, hatırlar herhalde" varsayımı en sık ve en pahalı hatadır. Hatırlamaz. Yazılı olan yaşar.

3 · Günlük akış — oturum döngüsü

Her gün aynı basit ritüel: aç, çalış, kapat. Çoğu otomatik.



Sabah (otomatik + 1 cümle sen)

Claude açılınca dünün özetini kendi okur. Sen sadece **günaydın** de — "şuradan kalmıştık, devam mı?" der.

Gün içi (esas iş burada)

- İş iste: "sipariş ekranına filtre ekle"
- Claude planını gösterir → sen **onayla** → yapar
- İş bitince **commit** iste: "bunu commit'le" (kaydetme noktası)
- Uyarı gelirse dinle: "15 dosya birikti, önce commit" gibi

Akşam (1 cümle)

iyi geceler yaz. Claude bugünü günlüğe yazar: ne bitti, ne kaldı, yarın nereden. Ertesi sabah bu sayede "hatırlar".

Tek cümlede döngü: Sabah **günaydın** , akşam **iyi geceler** . Arası senin işin. Hafıza otomatik taşınır.

Niçin bu kadar otomatik? Sabah özetini **session-start hook'u**, akşam günlüğünü **session-handoff skill'i** ve commit kaydını **post-commit-journal hook'u** yapıyor — yani **senin kurduğun yapılar**. Claude'u sihirli yapan bu dosyalar. Hiçbiri olmasa her sabah sıfırdan başlardın.

4 · Cheat-sheet — ne yazarsam ne olur

Buzdolabı kartı. Aklında tutman gereken tek sayfa.

Ne istiyorsan	Ne yaz / söyle
Güne başla, "nerede kaldık"	<code>günaydın</code>
Günü kapat, kaydet	<code>iyi geceler</code> · <code>/handoff</code>
İşi kaydet (kayıt noktası)	"bunu commit'le"
Çok dosya birikti, düzene sok	"dosyaları böl / commit-split"
Kararsızım, hangi seçenek?	"hangi seçenek?" · "council this"
Yazdığın kodu gözden geçir	<code>/code-check</code>
Güvenlik denetimi	<code>/security-check</code>
Değişikliği gerçekten çalıştır, doğrula	<code>/verify</code>
Yeni özelliği mimariyle kur	<code>/feature-dev <açıklama></code>
Kalıcı kural koy	"şunu kurala yaz: ..."
Çok kaynaklı araştırma	<code>/deep-research <soru></code>
Konuşma şişti, odakla	<code>/compact <konu></code>
Yeni/başka işe geç	<code>/clear</code>

Hatırlama yükü sıfır: Bunları ezberlemek zorunda değilsin. Düz Türkçe "şunu yapmak istiyorum" de — Claude doğru komutu/aracı kendi seçer. Bu tablo sadece hızlandırır.

Slash (/) komutu nedir?

`/` ile başlayanlar hazır **kısayol akışlardır**. `/` yazınca liste açılır, seçersin. Doğal dille de aynısını tetikleyebilirsin — örn. `/handoff` yerine "iyi geceler".

İki kafa karıştıran kavram, basitçe:

- **/compact** = "aynı işe devam ama konuşma uzadı, özetle." Hafıza korunur.
- **/clear** = "bambaşka işe geçiyorum, sıfırla." Önce `iyi geceler` demeyi unutma yoksa not kalmaz.

5 · Gerçek örnek oturum (baştan sona)

Bir günü olduğu gibi taklit et. Sol: sen. Sağ: ne olur.

Sen yazarsın	Olan
günaydın	Claude dünkü günlüğü okur: "Dün giriş formu bitti. Bugün 'şifremi unuttum' akışı sırada. Başlayalım mı?"
"evet, başla"	Planı gösterir: 3 dosya değişecek (form + servis + e-posta). "Onaylıyor musun?"
"onaylıyorum"	Dosyaları yazar. Her dosyada ne değişti gösterir.
"çalışıyor mu test et"	/verify mantığı: uygulamayı çalıştırır, akışı dener, sonucu söyler.
"güzel, commit'le"	Kayıt noktası atar: feat: şifre sıfırlama akışı
"e-posta metni hep Türkçe olsun"	"Bunu kurala yazayım mı?" → evet → kalıcı kural dosyasına işler.
"iyi geceler"	Günlüğe yazar: bitenler, yarına kalan (test yazımı), commit listesi. "Kaydedildi, iyi geceler."

Skill/kural YOKKEN

Her gün "biz Türkçe e-posta yolluyoruz, şifre şöyle sıfırlanır..." diye baştan anlatırsın. 10 dk/gün kayıp. Bir gün unutursun → İngilizce e-posta gider.

Kural VARKEN

Bir kez "kurala yaz" dedin. Artık Claude her oturum biliyor. Anlatım 0 dk. Tutarlılık garantili.

Gözlem: Hiçbir yerde kod yazmadın. Hedef söyledin, onayladın, kaydettirdin. Karar sende, yazım Claude'da.

5b · Ekran ekran — aynı oturum görsel

Yukarıdaki diyalog, terminalde adım adım böyle görünür.

```
proje

$ cd D:\Dev\proje
$ claude

Başlatılıyor... SessionStart
hook çalıştı ✓
```

1 Aç: `cd` + `claude`

```
Oturum Özeti
├─ Dün: giriş formu bitti
├─ Bugün: şifre sıfırlama
└─ Uncommitted: 3 ✓

> günaydın
```

2 Otomatik özet + `günaydın`

```
Plan: 3 dosya değişecek
- form.cshtml
- ResetService.cs
- Mail.cs

Onaylıyor musun? (e/h) > e
```

3 Plan gösterilir → onayla

```
ResetService.cs · diff

- public void Reset() {}
+ public async Task Reset(u){
+   var t = NewToken();
+   await Mail.Send(u, t); }

3 dosya güncellendi
```

4 Değişiklik diff'i (yeşil/kırmızı)

```
> çalışıyor mu test et
/verify → uygulama çalıştı
✓ şifre maili gönderildi
✓ 8/8 test geçti

Akış doğrulandı.
```

5 /verify — gerçekten çalıştı

```
> commit'le, iyi geceler
✓ feat: şifre sıfırlama
Journal yazıldı:
bitti · yarın: test yaz

İyi geceler 🌙
```

6 Commit + iyi geceler → kayıt

6 ekran = 1 gün. Aç → özet → onay → diff → doğrula → kaydet. Hep aynı ritim.

⚙ Bu ekranlardaki "sihir" nereden geliyor?

Dikkat — en önemli nokta: Bu otomatik davranışların hiçbiri Claude'un kendiliğinden yaptığı şey değil. Her biri **senin .claude/ altına yazdığın / Claude'a yazdırdığın skill, agent ve hook'lar sayesinde** çalışıyor. Sistem bu kadar akıcıysa, sebebi sensin.

Ekran	Görünen davranış	Neden çalışıyor (senin kurduğun yapı)
1-2	Açılıшта otomatik özet	session-start hook + session-protocol kuralı sayesinde
3	Önce plan + onay istemesi	plan-first kuralı sayesinde
4	Düzenli, tek-konu commit	commit-discipline kuralı sayesinde
5	"Bitti" demeden gerçekten test	verify skill + test-discipline kuralı sayesinde
6	Günlüğe yazıp kaydetmesi	session-handoff skill + post-commit-journal hook sayesinde

Çıkarım: "Claude akıllı" değil — **senin kurduğun yapı akıllı**. Yeni bir kolaylık istiyorsan: yeni bir skill/agent/hook yaz (veya yazdır). İleri seviyeler (Senior+) tam bunu öğretir.

Örnek: **günaydın** deyince tam olarak ne oluyor?

Tek kelime yazarsın, arkada **senin kurduğun yapı** şunları yapar:



Yani: "günaydın" sihirli değil. **session-start hook'u** dünkü durumu toplar, **session-handoff skill'i** (ters yön) son günlüğü okuyup özetler. Bu iki dosya olmasa "günaydın" hiçbir şey yapmazdı.

Hangi söz → hangi yapı → ne yapar

Sen dersin	Hangi yapı sayesinde	Ne yapar
günaydın	session-start hook + session-handoff	git/TODO/journal okur, "nerede kaldık" özetler
iyi geceler / /handoff	session-handoff skill	bugünü journale yazar, TODO işaretler, journal+TODO commit'ler
"bunu commit'le"	commit-discipline kuralı	tek-konu, doğru formatlı commit (feat: ...)
"dosyaları böl"	commit-splitter agent	15+ dosyayı anlamlı commit bucket'larına ayırır
"hangi seçenek?"	llm-council skill	5 danışman + sentez ile karar analizi
/verify	verify skill + test-discipline	uygulamayı çalıştırır, davranışı doğrular
(commit'te şifre)	pre-commit-antipattern hook	sızıntıyı yakalar, commit'i bloklar
(15 dosya birikince)	session-start + commit-discipline	"yeni iş yasak, önce böl" uyarısı

Kalıp: her kolaylık = **bir tetik söz + senin kurduğun bir yapı**. Yapıyı sil → kolaylık kaybolur. Yeni yapı yaz → yeni kolaylık kazan.

6 · Bir şey ters gitti — kurtarma

Panik yok. Çoğu sorun tek cümleyle çözülür.

Belirti	Ne yap
Claude yanlış dosyayı değiştirdi	"son değişikliği geri al" de. Henüz commit yoksa kolayca geri alır. Git her şeyi tutar — kalıcı kayıp nadirdir.
Commit "bloklandı" dedi	İyi haber — koruma çalıştı (örn. şifre sızıntısı yakalandı). Claude'a "neden bloklandı?" sor, düzelt, tekrar commit.
Cevaplar saçmaladı / konu dağıldı	Konuşma şişmiş. <code>/compact</code> (aynı iş) veya <code>/clear</code> (yeni iş). Önce iyi geceler ile notu kaydet.
Claude eski/yanlış bilgiyle konuşuyor	Hafıza eskimiş olabilir. "git durumunu tekrar oku" veya oturumu kapat-aç. Doğru kaynak: dosyalar, ezber değil.
"15 dosya eşiği" uyarısı	Çok iş birikti, kaydetmeden devam riskli. "dosyaları böl ve commit'le" de.
Bir kuralı sürekli unutuyor	Kural dosyada değil, konuşmada kalmış. "bunu kalıcı kurala yaz" de.
Yanlışlıkla bir şey sildim/bozdum	"git ile en son commit'e dön" de — ama önce Claude sana neyin kaybolacağını söyler, onayını ister.

Tehlikeli komutlar koruma altında: Geri alınamaz git işlemleri (`reset --hard` , `push --force`) Claude tarafından **onayın olmadan çalıştırılmaz**. "N dosya kaybolacak, emin misin?" diye sorar. Acele etme, oku, sonra onayla.

Genel kurtarma refleksi: Ne olduğunu Claude'a anlat ("şunu yaptım, böyle oldu"). Tanı + çözüm önerir. Bilmiyorsan tahmin etme — sor.

7 · Sınırlar, güven, SSS

Claude neyi yapar / yapmaz

YAPAR

- Kod okur, yazar, çalıştırır
- Önemli adımda onay ister
- İstersen commit eder
- Kurallara uyar
- Hata bulur, açıklar

YAPMAZ (izinsiz)

- Sen istemeden commit
- Geri alınamaz git komutu
- Sırları koda gömme
- Onaysız dosya silme
- "Bitti" demeden doğrulamadan geçme

Güvenlik temeli

- **Sırlar** (şifre, API anahtarı) koda **yazılmaz** — ortam değişkenine gider. Koruma bunu commit'te yakalar.
- Kullanıcıya teknik hata detayı gösterilmez; log'a gider.
- Her büyük değişiklik öncesi/sonrası `/security-check` iyi alışkanlık.

Sık sorulan acemi soruları

Soru	Cevap
Benim yerime kod mu yazıyor?	Evet ama kontrol sende — onaylamadan kalıcı olmaz. Sen mimar, o uygulayıcı.
Terminali kapatınca kaybolur mu?	Konuşma kaybolur ama <code>iyi geceler</code> dediysen önemli her şey dosyaya yazıldı. Commit'lediysen kod güvende.
Kod bilmem, kullanabilir miyim?	Evet. Ne istediğini Türkçe anlat yeter. Ama ürettiğini gözden geçirmek için temel bilgi faydalı.
Yanlış yaparsa kim sorumlu?	Sen onaylıyorsun, sen sorumlusun. O yüzden gösterdiğini oku, körü körüne "onayla" deme.
Skill/agent/hook ne farkı?	Karıştırma — "şunu istiyorum" de, doğru olanı Claude seçer + gerekçesini söyler.

Mini sözlük (jargon)

`oturum` = bir `claude` açıp kapatma süresi. `bağlam (context)` = Claude'un o an "aklındaki" bilgi.
`commit` = kod kaydetme noktası (git). `journal` = günlük not defteri dosyası. `kural (rule)` = kalıcı davranış talimatı. `skill` = hazır iş reçetesi. `agent` = ayrı kafada iş yapan yardımcı. `hook` = otomatik tetiklenen kontrol. `compact` = konuşmayı özetle-sıkıştır. `clear` = sıfırla, yeni iş.
`handoff` = gün sonu devir/kaydetme. `tier` = işin büyüklük seviyesi (plan gerekir mi).

Tek cümlelik özet: Ne istediğini söyle · gösterdiğini oku · onayla · kaydettir. Gerisini sistem halleder.

claude-context-template · Acemi Rehberi · Daha fazlası: GELISTIRICI_REHBERI.md, PROJE_OZEL_KAVRAM.md